

Bloodrose — Story Forge

Integration Layer

Governed Cross-System Integration Specification

Project	Bloodrose
Artifact	Story Forge Integration Layer
Version	v0.1
Classification	Governed Cross-System Integration
Date	April 21, 2026
Status	Active
Depends On	Bloodrose Canonical Spec v0.1, Bloodrose Modular Architecture v0.1, Story Forge Platform (external)

1. Integration Philosophy

Bloodrose is not a standalone game. It is one render target of a broader narrative system. The story of Lady Isolde — her betrayal, fragmentation, rage, mercy, and self-reclamation — exists as a canonical story spine that is medium-independent. The game prototype is one way to experience it. A visual novel, a cinematic trailer, a comic are others. Story Forge is the system that holds the canonical spine and feeds it to each render target.

The Integration Layer defines the contract between Story Forge (the narrative authority) and Bloodrose (the interactive render target). Story Forge provides: canonical narrative content, character definitions, emotional arc structures, world rules, and event sequences. Bloodrose provides: interactive state, player-driven progression, real-time emotional mechanics, and session data that feeds back into Story Forge for analysis and cross-media coordination.

The relationship is governed. Story Forge does not inject arbitrary content into Bloodrose — it provides content through defined channels that Bloodrose's own governance (Event Table, Drift Detection Harness, Operator Surface) validates before acceptance. Bloodrose does not arbitrarily generate narrative — its narrative outputs are sourced from or validated against the canonical spine held by Story Forge.

This is a bidirectional integration. Downstream: Story Forge feeds narrative content into Bloodrose. Upstream: Bloodrose feeds session data back to Story Forge. The Integration Layer governs both directions.

2. Conceptual Model

The following concepts structure the integration between Story Forge and Bloodrose. Each term has a precise, bounded definition that governs how data flows between systems.

2.1 Story Spine

The medium-independent narrative backbone. The story spine is the canonical source of truth for what the story *is*, independent of how any particular medium expresses it. For Bloodrose, the story spine includes:

- **Character identity:** Lady Isolde — betrayed bloodwitch, emotionally fractured, trapped in the heart-garden
- **Core conflict:** corruption vs. redemption
- **Narrative axis:** trauma, desire, rage, mercy, self-worth
- **Setting:** The Heart-Garden — a surreal, living inner realm shaped by betrayal, memory, and magical consequence
- **Canonical endings:** Consumed by Thorns (loss), Mercy Dominant (win A), Bound but Unbroken (win B)
- **Thematic invariants:** betrayal, emotional fragmentation, toxic desire, rage, guilt, mercy, choosing oneself after damage

Governance Constraint

The story spine is owned by Story Forge. Bloodrose consumes it. Bloodrose may not modify, extend, or contradict any element of the story spine without an amendment flowing through Story Forge.

2.2 Render Target

A medium-specific expression of the story spine. Bloodrose is a render target of type "interactive game." Each render target has:

- **A medium-specific state model** — Bloodrose: corruption/redemption/score
- **Medium-specific mechanics** — Bloodrose: movement, collection, hazards, altar
- **Medium-specific presentation** — Bloodrose: top-down game with meters, narrative log, win/lose screens
- **A mapping layer** that translates the story spine into medium-specific constructs

2.3 Narrative Event

A story beat defined in the story spine that can be expressed across render targets. In Bloodrose, narrative events map to the Canonical Event Table (EVT-IDs). In a visual novel, the same narrative event might map to a dialogue scene. In a cinematic trailer, it might map to a visual sequence. The Integration Layer ensures the same narrative event is recognizable across all render targets.

2.4 Canon Lock

A governance mechanism that prevents drift between Story Forge and its render targets. When a narrative element is "canon-locked," it cannot be modified by the render target without an amendment flowing through Story Forge. Canon locks apply to: character identity, core conflict, thematic invariants, ending conditions, and emotional arc structure. Lock levels (hard and soft) are defined in Section 7.

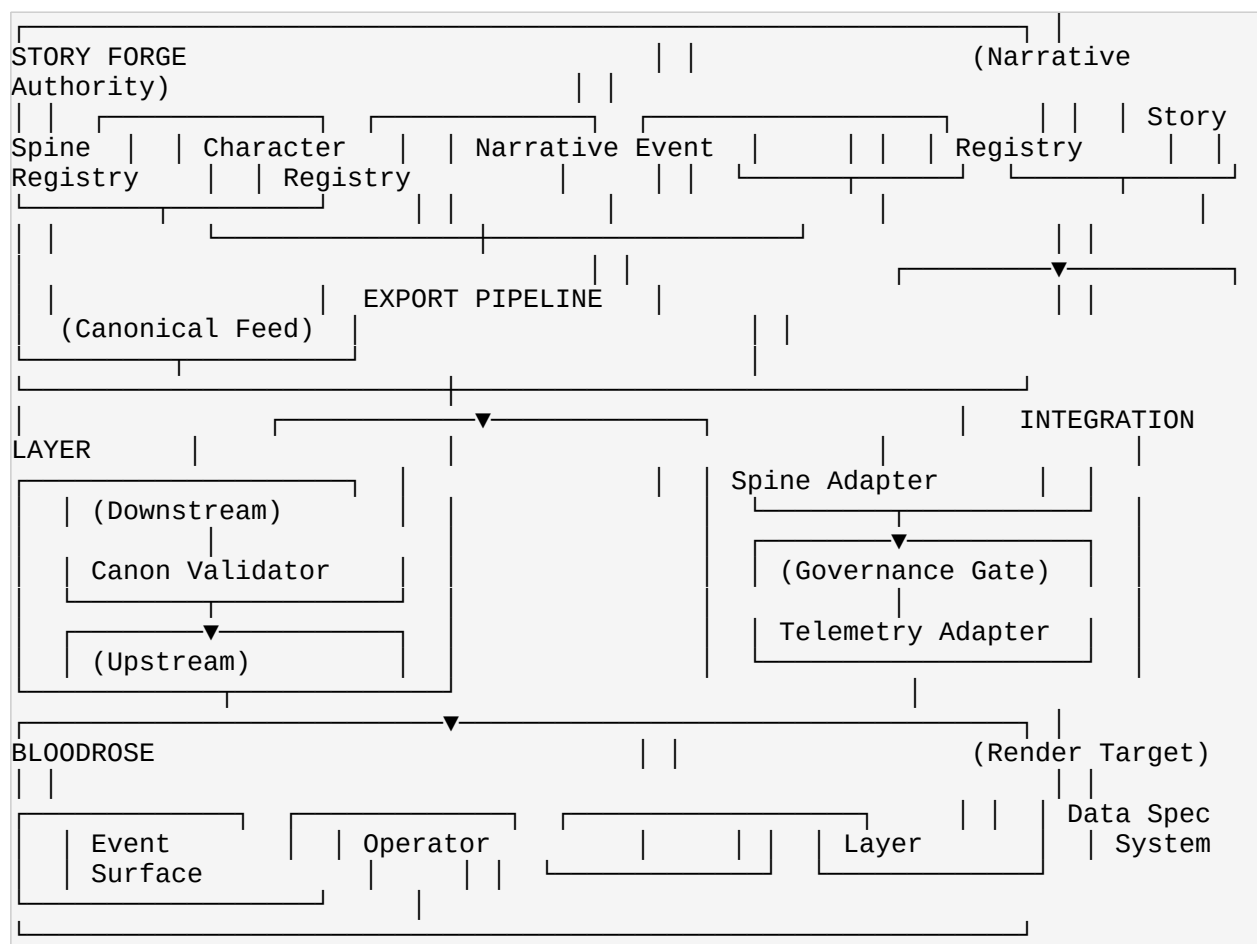
2.5 Session Telemetry

The data Bloodrose sends back to Story Forge after a session. This includes: state trajectory (corruption/redemption over time), events fired, ending reached, player behavior patterns. Story Forge uses this data for cross-media coordination and narrative analytics.

3. Integration Architecture

3.1 Architecture Diagram

The following diagram illustrates the structural relationship between Story Forge, the Integration Layer, and Bloodrose:



3.2 Downstream Flow (Story Forge → Bloodrose)

The following content flows from Story Forge into Bloodrose through the Integration Layer's downstream path:

- **Character definitions:** Lady Isolde's identity, motivation, emotional axis, narrative voice. These feed into Bloodrose's narrative text bank (Data Spec Layer) and ensure narrative output matches canonical character.
- **Narrative event definitions:** Story beats with associated emotional tone, trigger conditions (medium-independent), and canonical text variants. These map to Bloodrose's Canonical Event Table entries.
- **World rules:** The Heart-Garden's symbolic logic — what thorns represent, what petals represent, what the altar represents. These inform Bloodrose's entity design and hazard logic.
- **Emotional arc structure:** The progression from fragmentation through conflict to resolution (or consumption). This maps to Bloodrose's threshold events and ending conditions.
- **Ending definitions:** The canonical endings with their thematic meaning and trigger conditions. These map to Bloodrose's terminal events.

3.3 Upstream Flow (Bloodrose → Story Forge)

The following data flows from Bloodrose back to Story Forge through the Integration Layer's upstream path:

- **Session telemetry:** Corruption/redemption trajectories, events fired, ending reached, session duration, petal collection patterns.
- **Player behavior analytics:** Which emotional collectibles were sought vs. avoided? How long did players spend in thorn zones? How many attempts before reaching the altar?

- **Narrative effectiveness data:** Which narrative log entries appeared in winning sessions vs. losing sessions? Which emotional beats correlated with player engagement?
- **Cross-media coordination data:** If a player experienced the Mercy Dominant ending in the game, Story Forge can coordinate the visual novel to reference or build on that experience.
- **Drift reports:** Bloodrose's Drift Detection Harness feeds compliance data to Story Forge so the narrative authority knows the render target is behaving canonically.

4. Integration Components

4.1 Spine Adapter (Downstream)

Purpose: Translates Story Forge's medium-independent narrative data into Bloodrose-specific formats.

Responsibilities:

- Receives canonical narrative content from Story Forge's Export Pipeline
- Transforms character definitions into Bloodrose's narrative text bank format
- Transforms narrative event definitions into Bloodrose's Event Table entry format (EVT-ID, trigger condition, state effects, narrative text, priority, repeatability)
- Transforms world rules into Bloodrose's Data Spec configuration values
- Validates incoming content against Bloodrose's architectural constraints (e.g., ensures no event references undefined state variables)
- Stages transformed content for Canon Validator review before injection into Bloodrose

Interface:

- **Input:** StoryForgeExport (JSON) — contains character_defs, narrative_events, world_rules, arc_structure, endings
- **Output:** BloodrosePatch (JSON) — contains data_spec_updates, event_table_entries, narrative_text_updates

Governance Constraint

The Spine Adapter never directly modifies Bloodrose modules. It produces a patch that must pass through the Canon Validator. No bypass path exists.

4.2 Canon Validator (Governance Gate)

Purpose: Ensures that content flowing from Story Forge into Bloodrose does not violate Bloodrose's canonical constraints. This is the governance gate — nothing passes without validation.

Responsibilities:

- Validates that all event entries in the patch have valid trigger conditions referencing known state variables
- Validates that state effects are bounded (no corruption delta that could exceed 0-100 range in a single application)
- Validates that narrative text matches canonical aesthetic tone (automated: keyword presence; manual: human review flag for subjective tone)
- Validates that no patch introduces entities, mechanics, or state variables outside the canonical spec without an accompanying amendment
- Cross-references the patch against the Drift Detection Harness's Canonical Validation Rules (CVR table)
- Produces a validation report: **PASS** (all checks clear), **CONDITIONAL** (warnings but no blockers), **REJECT** (critical violations found)

Interface:

- **Input:** `BloodrosePatch` from Spine Adapter
- **Output:** `ValidationReport` (JSON) — containing status (pass/conditional/reject), `violations` list, `warnings` list, `approval_required` flag

Validation Result	Action
PASS	Patch is staged for application to Bloodrose modules
CONDITIONAL	Patch is staged with warnings; operator review recommended via Operator Surface
REJECT	Patch is blocked; violations must be resolved in Story Forge before re-export

4.3 Telemetry Adapter (Upstream)

Purpose: Packages Bloodrose session data into a format Story Forge can consume for cross-media coordination and narrative analytics.

Responsibilities:

- Collects session summary data from the Operator Surface's Session Summary Panel
- Collects state trajectory data (corruption/redemption snapshots over time)
- Collects event log summary (events fired, categories, frequencies)
- Collects drift report summary from the Drift Detection Harness
- Packages all data into a `StoryForgeTelemetry` payload
- Transmits payload to Story Forge's Ingestion Pipeline (when available)
- In v0.1, generates the payload as a local JSON export for manual import into Story Forge

Interface:

- **Input:** `SessionSummary`, `StateTrajectory`, `EventLog`, `DriftReport` from Bloodrose modules

- **Output:** StoryForgeTelemetry (JSON) — containing session_id, render_target ("bloodrose_game"), ending_type, state_trajectory, event_summary, drift_status, behavioral_analytics
- **Transmission:** Local export in Phase 1; API push in Phase 3

5. Data Contracts

The following schemas define the formal data structures exchanged between Story Forge and Bloodrose. All payloads are JSON. All fields are required unless marked *optional*.

5.1 StoryForgeExport Schema

StoryForgeExport

```

├─ spine_id : string – Unique identifier for the story spine
├─ spine_version : string – Semantic version of the story spine being exported
├─ character_definitions : array
|   ├─ id : string – Character unique identifier
|   ├─ name : string – Character canonical name
|   ├─ role : string – Narrative role (e.g., "protagonist")
|   ├─ emotional_axis : array of strings – Primary emotional dimensions
|   ├─ narrative_voice_keywords : array of strings – Tone-defining keywords
|   └─ thematic_invariants : array of strings – Non-negotiable thematic elements
├─ narrative_events : array
|   ├─ spine_event_id : string – Cross-media event identifier
|   ├─ event_name : string – Human-readable event name
|   ├─ emotional_category : string – (guilt / desire / rage / mercy / neutral)
|   ├─ trigger_description_abstract : string – Medium-independent trigger condition
|   ├─ canonical_text_variants : object – Keyed by render target type
|   └─ thematic_weight : number – 0.0–1.0 narrative significance
├─ world_rules : array
|   └─ rule_id : string – Rule unique identifier

```

```

|   ├── rule_description : string – Human-readable rule statement
|   ├── symbolic_mapping : string – What the rule represents thematically
|   └── mechanical_implication : string – How the rule constrains render targets
└── arc_structure : object
    ├── phases : array
    |   ├── phase_name : string
    |   ├── emotional_state : string
    |   ├── threshold_description : string
    |   └── narrative_markers : array of strings
    ├── endings : array
    |   ├── ending_id : string – Ending unique identifier
    |   ├── ending_name : string – Canonical ending name
    |   ├── thematic_meaning : string – What this ending signifies
    |   ├── trigger_conditions_abstract : string – Medium-independent trigger
    |   └── canonical_resolution_text : string – Canonical concluding text
    └── canon_locks : array
└── element_type : string – (character / setting / conflict / ending / axis)
└── element_id : string – Reference to locked element
└── lock_level : string – "hard" or "soft"
└── reason : string – Justification for the lock

```

5.2 BloodrosePatch Schema

BloodrosePatch

```

└── patch_id : string – Unique patch identifier
└── source_spine_id : string – Reference to originating story spine
└── source_spine_version : string – Version of the source spine
└── data_spec_updates : array
    ├── key : string – Config key to update
    ├── value : any – New value
    └── source_rule_id : string – Traceability to originating world rule
└── event_table_entries : array

```

- | └─ event_id : string – Bloodrose EVT-ID format
- | └─ event_name : string
- | └─ category : string – Event category
- | └─ trigger_condition : string – Bloodrose-specific trigger
- | └─ state_effects : object – { corruption_delta, redemption_delta, score_delta }
- | └─ narrative_output : string – Narrative log text
- | └─ repeatable : boolean
- | └─ priority : number
- | └─ notes : string (*optional*)
- └─ narrative_text_updates : array
 - | └─ event_id : string – Target event
 - | └─ field : string – Field to update
 - | └─ old_text : string – Previous text (for audit trail)
 - | └─ new_text : string – Replacement text
- └─ entity_definitions : array (*optional*)
 - | └─ Entity schema updates – requires amendment if new entities introduced
 - └─ requires_amendment : boolean – True if patch introduces new state variables, entities, or mechanics not in the current canonical spec

5.3 StoryForgeTelemetry Schema

StoryForgeTelemetry

- └─ session_id : string – Unique session identifier
- └─ render_target : string – "bloodrose_game"
- └─ render_target_version : string – "0.1"
- └─ timestamp : string – ISO 8601 format
- └─ session_duration_seconds : integer
- └─ ending_type : string – consumed / mercy_dominant / bound_unbroken / null
- └─ state_trajectory : array
 - | └─ tick : integer
 - | └─ corruption : number (0–100)

```

|   └─ redemption : number (0-100)
|   └─ score : integer
└─ event_summary : object
|   └─ total_events : integer
|   └─ by_category : object - { category: count }
|   └─ by_event_id : object - { event_id: count }
└─ petal_collection : object
|   └─ guilt_count : integer
|   └─ desire_count : integer
|   └─ rage_count : integer
|   └─ mercy_count : integer
└─ thorn_contact : object
|   └─ episodes : integer
|   └─ total_corruption_from_thorns : number
└─ altar_interactions : object
|   └─ attempts : integer
|   └─ rejections : integer
|   └─ success : boolean
└─ drift_status : string - clean / warnings / critical_drift
└─ drift_violations : array (optional)
└─ Violation summary strings

```

6. Canon Lock Specification

Canon locks are the primary governance mechanism preventing drift between Story Forge and its render targets. This section defines lock levels and the canonical lock registry for Bloodrose.

6.1 Lock Levels

Hard Lock

The element cannot be modified by the render target under any circumstances. Changes must originate from Story Forge and flow downstream through the Integration Layer. Hard-locked elements include:

- Lady Isolde's identity (name, role, core conflict)
- The Heart-Garden as setting (symbolic meaning of thorns, petals, altar)
- Canonical endings (the three ending types and their thematic meaning)
- Narrative axis (trauma, desire, rage, mercy, self-worth)

Soft Lock

The element has a canonical default provided by Story Forge, but the render target may adapt it for medium-specific needs. Adaptations must be logged and reported upstream via the Telemetry Adapter. Soft-locked elements include:

- Specific narrative text (wording may be adapted for interactive pacing)
- Collectible effect values (deltas may be tuned for gameplay balance, but emotional polarity must be preserved)
- Visual aesthetic details (crimson/black/gothic palette is hard-locked; specific sprite designs are soft-locked)
- Pacing thresholds (score milestones may be adjusted for session length)

Important

Soft-lock adaptations must preserve emotional polarity. A collectible that canonically increases corruption cannot be adapted to decrease it, regardless of gameplay balance arguments. Polarity inversion is treated as a hard-lock violation.

6.2 Lock Registry

Element	Element ID	Lock Level	Owner	Canon Reference	Notes
Lady Isolde Identity	CHAR-001	Hard	Story Forge	Spec Section 4	Name, role, core conflict immutable
Heart-Garden Setting	WORLD-001	Hard	Story Forge	Spec Section 5	Symbolic meaning of all setting elements
Corruption/Redemption Axis	AXIS-001	Hard	Story Forge	Spec Section 7	Dual-axis structure is canonical
Ending: Consumed by Thorns	END-001	Hard	Story Forge	Event Table EVT-030	Loss condition — corruption overwhelms
Ending: Mercy Dominant	END-002	Hard	Story Forge	Event Table EVT-031	Win A — redemption triumph
Ending: Bound but Unbroken	END-003	Hard	Story Forge	Event Table EVT-032	Win B — morally complex resolution
Thorn Corruption-Only Rule	RULE-001	Hard	Story Forge	Spec Section 9	Thorns only increase corruption, never decrease
Redemption Threshold for Altar (65)	RULE-002	Hard	Story Forge	Spec Section 10	Minimum redemption required for altar acceptance
Narrative Text (EVT-001-004)	TEXT-001	Soft	Story Forge	Event Table	Wording adaptable for interactive pacing
Petal Effect	MECH-001	Soft	Bloodrose	Spec	Polarity

Element	Element ID	Lock Level	Owner	Canon Reference	Notes
Deltas				Section 8	constraint from Story Forge enforced
Score Milestone Thresholds	MECH-002	Soft	Bloodrose	Event Table EVT-024	Adjustable for session length tuning
Thorn Corruption Rate (per tick)	MECH-003	Soft	Bloodrose	Event Table EVT-010	Rate tunable; polarity (increase only) is hard-locked

7. Cross-Media Coordination Protocol

Story Forge uses Bloodrose telemetry — and telemetry from all other render targets — to coordinate narrative expression across media. The following capabilities describe how this coordination operates.

7.1 Narrative Echo

When a player completes a Bloodrose session with a specific ending, Story Forge can "echo" that narrative outcome into other render targets. Examples:

- A player who reached the **Mercy Dominant** ending in the game might encounter a visual novel scene that references "the day the thorns retreated"
- A player who was **Consumed by Thorns** might encounter a cinematic sequence that opens with the garden closing
- A player who achieved **Bound but Unbroken** might encounter a comic panel showing Isolde scarred but standing — acknowledging the damage without erasing it

Phase Note

Narrative Echo is a future capability (Phase 3+). However, the telemetry structure defined in this specification must support it from v0.1. The `ending_type`, `state_trajectory`, and `petal_collection` fields in `StoryForgeTelemetry` provide the data foundation.

7.2 Behavioral Insight

Story Forge analyzes player behavior across sessions to inform narrative design across all render targets:

- If most players avoid Guilt petals, the narrative engine may adjust future content to explore guilt more aggressively in other media
- If players consistently reach the altar with very high corruption (Bound but Unbroken), this signals audience affinity for morally complex resolutions
- If thorn avoidance dominates player behavior, future render targets may design more inescapable confrontations with corruption

7.3 Canon Consistency Monitoring

Story Forge tracks whether all render targets are expressing the story spine consistently. Key monitoring questions:

- Are all render targets using the same character identity for Lady Isolde?
- Are ending conditions thematically aligned across media?
- Has any render target drifted from the canonical narrative axis?
- Are emotional polarities preserved across all expressions of the story?

The Integration Layer's upstream telemetry — including drift reports from the Drift Detection Harness — feeds this monitoring. Story Forge aggregates drift status across all connected render targets and flags cross-media inconsistencies.

8. Implementation Phases

8.1 Phase 1 — Prototype (v0.1)

- Integration Layer is conceptual only — no live connection between Story Forge and Bloodrose
- Bloodrose's canonical documents (Canonical Spec, Event Table, Architecture Document) serve as the de facto story spine
- Telemetry is manual: session data exported as JSON from Operator Surface, manually reviewed
- Canon locks are enforced by the Contractor Contract and Drift Detection Harness, not by automated Story Forge integration
- The Spine Adapter, Canon Validator, and Telemetry Adapter are documented specifications — not running code
- Data contract schemas are defined and frozen for forward compatibility

8.2 Phase 2 — Modular

- Spine Adapter implemented as a build-time tool: reads a Story Forge export file and generates Bloodrose Data Spec and Event Table updates
- Canon Validator implemented as a pre-commit check integrated with the Drift Detection Harness
- Telemetry Adapter implemented: generates StoryForgeTelemetry JSON after each session
- Canon Lock Registry maintained as a versioned document; soft-lock adaptations logged with rationale
- Patch application becomes semi-automated: Spine Adapter produces patch, Canon Validator approves, operator applies

8.3 Phase 3 — Production Pipeline

- Live API connection between Story Forge and Bloodrose
- Spine Adapter processes real-time updates from Story Forge's Export Pipeline
- Canon Validator runs automatically on every Story Forge export
- Telemetry streams from Bloodrose to Story Forge in real-time or near-real-time
- Cross-media coordination active: Narrative Echo, Behavioral Insight, Canon Consistency Monitoring
- Story Forge operator dashboard shows Bloodrose as a connected render target with live status
- Disconnected mode supported for resilience (see Section 10, Error Handling)

Capability	Phase 1	Phase 2	Phase 3
Spine Adapter	Spec only	Build-time tool	Real-time pipeline
Canon Validator	Manual (Drift Harness)	Pre-commit check	Automated gate
Telemetry Adapter	Manual JSON export	Auto-generated JSON	Streaming API
Canon Locks	Contract-enforced	Versioned document	Machine-enforced
Cross-Media Coordination	Not available	Not available	Active
Narrative Echo	Not available	Not available	Active

9. Governance Rules

The following rules govern the Integration Layer. They are binding on all operators, contributors, and automated systems interacting with the Story Forge-Bloodrose boundary.

Rule 1 — Narrative Authority

Story Forge is the narrative authority. Bloodrose cannot override hard-locked canonical elements. Any modification to a hard-locked element must originate from Story Forge and flow downstream through the Integration Layer.

Rule 2 — Dual Governance

The Integration Layer must never bypass Bloodrose's internal governance (Drift Detection Harness, Operator Surface, Event Table validation). Story Forge content is validated by Bloodrose's own systems before acceptance. Two governance systems must agree — neither can override the other.

Rule 3 — Polarity Preservation

Soft-lock adaptations must preserve emotional polarity. A collectible that canonically increases corruption cannot be adapted to decrease it, regardless of gameplay balance arguments. A petal that canonically grants mercy cannot be repurposed to express rage. Polarity is thematic truth — it is not a tuning parameter.

Rule 4 — Privacy First

Upstream telemetry must not include player-identifying information. Session data is anonymized at the Telemetry Adapter before export. No biometric data. No personal identifiers. No behavioral profiles tied to individuals. Telemetry serves narrative design, not surveillance.

Rule 5 — Governed Document

The Integration Layer specification itself is a governed document subject to the amendment process defined in the Contractor Contract. Changes to data contracts, canon locks, or governance rules require versioned amendments with review and approval.

Rule 6 — Bidirectional Skepticism

"Suggestion is not truth" applies bidirectionally. Story Forge suggestions do not become Bloodrose canon without passing through the Canon Validator. Bloodrose telemetry observations do not become Story Forge canon without narrative authority review. Both systems maintain epistemic discipline.

10. Error Handling

The Integration Layer must handle failures gracefully, preserving data integrity and canonical compliance. All error states produce operator-visible alerts through the Operator Surface.

Failure Scenario	Severity	Response
Spine Adapter receives malformed Story Forge export	ERROR	Log error, reject entire import, notify operator. No partial import permitted.
Canon Validator finds critical violation in patch	REJECT	Reject the entire patch. Atomic acceptance or atomic rejection — no partial patches.
Telemetry Adapter fails to generate payload	WARNING	Store raw session data for retry. Session data must not be lost under any circumstance.
Story Forge API	WARNING	Bloodrose continues

Failure Scenario	Severity	Response
unavailable (Phase 3)		independently using last-known canonical documents. Integration Layer enters "disconnected mode" and queues telemetry for later transmission.
Version mismatch between Story Forge spine version and Bloodrose's last-imported version	WARNING	Canon Validator flags mismatch. Operator must review and explicitly approve the update before application.

Critical Constraint

Partial patch application is never permitted. If a patch contains 15 event updates and one fails validation, all 15 are rejected. This prevents canonical state fragmentation where Bloodrose holds a mix of validated and unvalidated content.

11. Integration Points with Other Artifacts

The Story Forge Integration Layer connects to every major artifact in the Bloodrose production suite. The following table defines each integration point, the direction of data flow, and the nature of the dependency.

Artifact	Relationship	Integration Detail
Canonical Event Table (Artifact 1)	Downstream target	The Spine Adapter generates Event Table entries from Story Forge narrative events. The Canon Validator cross-references incoming patches against the

Artifact	Relationship	Integration Detail
		existing Event Table to detect conflicts, duplications, and coverage gaps.
Contractor Contract (Artifact 2)	Governance alignment	Contributors must understand that narrative content originates from Story Forge and is canon-locked. They cannot independently author narrative that contradicts the story spine. The Contract's amendment process governs changes to the Integration Layer itself.
Operator Surface Spec (Artifact 3)	Bidirectional	The Operator Surface displays Integration Layer status — last Story Forge sync timestamp, canon lock compliance status, telemetry export readiness. The Telemetry Adapter reads session data from the Operator Surface's Session Summary Panel.
Modular Architecture (Artifact 4)	Structural dependency	The Integration Layer feeds into the Data Spec Layer (downstream content) and reads from the Operator Surface and Event System (upstream telemetry). The 8-module architecture defines where Integration Layer outputs are consumed.
Drift Detection Harness (Artifact 5)	Bidirectional	The Canon Validator leverages the harness's assertion library (ASSERT-001 through ASSERT-010) to validate Story Forge patches. Drift reports generated by the harness flow upstream through the Telemetry Adapter to Story Forge.

Artifact	Relationship	Integration Detail
Render Pipeline Spec (Artifact 7, forthcoming)	Downstream consumer	The Render Pipeline consumes the same Data Spec that the Integration Layer populates. Aesthetic canon locks (crimson/black/gothic palette) constrain what the Render Pipeline can express visually.
Cross-Media Adaptation Spec (Artifact 8, forthcoming)	Protocol expansion	The cross-media coordination protocol defined in Section 7 of this document is expanded in Artifact 8 with full adaptation rules per medium, render-target-specific mapping tables, and Narrative Echo implementation details.

12. Amendment Log

All modifications to this specification are tracked below. Amendments follow the governance process defined in the Contractor Contract v0.1.

Amendment ID	Date	Description	Author	Approved By
AMD-000	April 21, 2026	Initial Story Forge Integration Layer created. Defines conceptual model, integration architecture, data contracts, canon lock specification, cross-media coordination	System Architect	Pending Review

Amendment ID	Date	Description	Author	Approved By
		protocol, implementation phases, governance rules, and error handling.		

*Bloodrose — Story Forge Integration Layer v0.1 — Governed Cross-System
Integration Specification*

This document is subject to the amendment process. Unauthorized modifications constitute canonical drift.